



XML et Services Web

M. KASSE

Université Alioune DIOP de BAMBEY

youssou.kasse@uadb.edu.sn

Master SI

2017



Le langage de définition de schéma XML

Le langage de définition de schéma XML

- XML Schéma est une recommandation du W3C de mai 2001 qui consiste à définir une méthodologie de description de document XML autrement que par les DTD.
- L'approche cherche à résoudre plusieurs problèmes des DTD dont l'épine dorsale est le fait qu'un DTD n'est pas un document XML.
- En conclusion, il est nécessaire de connaître un autre langage que XML pour décrire un DTD (le langage propre aux DTD) mais également de disposer d'autres outils pour éditer et manipuler des DTD (par exemple, des parseurs spéciaux).

Le langage de définition de schéma XML

- Un XML Schéma est donc un document dont l'objectif est similaire à un DTD mais sous forme d'un document en XML.
- Plus clairement, c'est un document XML qui décrit un document XML !
- C'est pour cette raison qu'on parle de méta données pour les données constituant un XML Schéma.
- XML Schema est un document XML qui utilise un ensemble de balises définies au sein de la recommandation du W3C.

Le langage de définition de schéma XML

- Le but d'un schéma est de définir une classe de documents XML.
- Il permet de décrire les autorisations d'imbrication et l'ordre d'apparition des éléments et de leurs attributs, tout comme une DTD. Mais il permet aussi d'aller au-delà.
- Un premier point intéressant est qu'un fichier Schema XML est un document XML.
- Cela permet à un tel document d'être manipulé de la même manière que n'importe quel autre fichier XML.

Structure de base

- Comme tout document XML, un Schema XML commence par un prologue, et a un élément racine.

```
<?xml version="1.0" encoding="ISO-8859-1"?>  
<xsd:schema xmlns:xsd="http://www.w3.org/2000/10/XMLSchema">  
  
    <!-- déclarations d'éléments, d'attributs et de types    ici -->  
  
</xsd:schema>
```

Déclarations d'éléments

- Un élément, dans un schéma, se déclare avec la balise **<xsd:element>**.

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2000/10/XMLSchema">
    <xsd:element name="contacts" type="typeContacts" />
    <xsd:element name="remarque" type="xsd:string" />
<!-- déclarations de types ici -->
</xsd:schema>
```

Chaque élément déclaré est associé à un type de données via l'attribut `type`. Les éléments pouvant contenir des élément-enfants ou posséder des attributs sont dits de type *complexe*, tandis que les éléments n'en contenant pas sont dits de type *simple*.

Déclarations d'attributs

- Contrairement aux éléments, un attribut ne peut être que de type simple.
- Cela signifie que les attributs, comme avec les DTD, ne peuvent contenir d'autres éléments ou attributs.
- De plus, les déclarations d'attributs doivent être placées *après* les définitions des types complexes

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2000/10/XMLSchema">
  <xsd:element name="contacts" type="typeContacts" />
    <xsd:element name="remarque">
      <xsd:complexType>
        <xsd:attribute name="datecom" type="xsd:date" />
      </xsd:complexType>
    </xsd:element>
</xsd:schema>
```


Contraintes d'occurrences

- Tout comme dans une DTD, un attribut peut avoir un indicateur d'occurrences.
- L'élément `attribute` d'un Schema XML peut avoir trois attributs optionnels : `use`, `default` et `fixed`.
- Des combinaisons de ces trois attributs permettent de paramétrer ce qui est acceptable ou non dans le fichier XML final (attribut obligatoire, optionnel, possédant une valeur par défaut...).

```
<xsd:attribute name="datecom" type="xsd:date" use="optional"
default="2003-10-11" />
```

Contraintes d'occurrences

- **Déclaration d'une valeur par défaut**

- `<xsd:attribute name="lang" type="xsd:string" default="EN"/>`

- **Déclaration d'une valeur fixe**

- `<xsd:attribute name="lang" type="xsd:string" fixed="EN"/>`

- **Déclaration d'un attribut optionnel**

- `<xsd:attribute name="lang" type="xsd:string" use="optional" />`

- **Déclaration d'un attribut obligatoire**

- `<xsd:attribute name="lang" type="xsd:string" use="required"/>`



Les types de donnée communs

- XML schema a beaucoup de types de données.
Voici la liste des types les plus commun
 - xsd:string
 - xsd:decimal
 - xsd:integer
 - xsd:boolean
 - xsd:date
 - xsd:time

Restriction dans les schémas

- Restriction de la valeur d'élément age entre 0 et 100

```
<xsd:element name="age">  
  <xsd:simpleType>  
    <xsd:restriction base="xsd:integer">  
      <xsd:minInclusive value="0"/>  
      <xsd:maxInclusive value="100"/>  
    </xsd:restriction>  
  </xsd:simpleType>  
</xsd:element>
```

Restriction dans les schémas

- Restriction sur l'ensemble des valeurs

```
<xsd:element name="car">
  <xsd:simpleType>
    <xsd:restriction base="xsd:string">
      <xsd:enumeration value="Audi"/>
      <xsd:enumeration value="Golf"/>
      <xsd:enumeration value="BMW"/>
    </xsd:restriction>
  </xsd:simpleType>
</xsd:element>
```

Restriction dans les schémas

- Restriction sur la série des valeurs

```
<xsd:element name="letter">
  <xsd:simpleType>
    <xsd:restriction base="xsd:string">
      <xsd:pattern value="[a-z]"/>
    </xsd:restriction>
  </xsd:simpleType>
</xsd:element>
```

Restriction dans les schémas

- Restriction sur la longueur

```
<xsd:element name="password">  
  <xsd:simpleType>  
    <xsd:restriction base="xsd:string">  
      <xsd:length value="8"/>  
    </xsd:restriction>  
  </xsd:simpleType>  
</xsd:element>
```

Restriction dans les schémas

- Restriction sur la longueur

```
<xsd:elementname="password">  
  <xsd:simpleType>  
    <xsd:restriction base="xsd:string">  
      <xsd:minLength value="5"/>  
      <xsd:maxLength value="8"/>  
    </xsd:restriction>  
  </xsd:simpleType>  
</xsd:element>
```


Élément Complexe

- L'élément complexe est un élément XML qui contient les autres éléments et/ou attributs
- Il y a 4 types d'éléments complexes:
 - Élément vide
 - Élément qui contient seulement d'autres éléments
 - Élément qui contient que des textes, avec éventuellement des attributs.
 - Élément qui contient d'autres éléments et du texte

Définir des types complexes pour des éléments vides

```
<personne ident="2505" />
```

```
<xsd:element name="personne">  
  <xsd:complexType>  
    <xsd:attribute name="ident" type="xsd:ID"/>  
  </xsd:complexType>  
</xsd:element>
```

Définir des types complexes pour des éléments mixtes

<b e="valeur">données

```
<xsd:element name="b">
  <xsd:complexType mixed="true">
    <xsd:attribute name="e" use="required" type="xsd:ID"/>
  </xsd:complexType>
</xsd:element>
```

Définir des types complexes pour un élément

```
<employee>  
  <firstname>Moussa</firstname>  
  <lastname>Bâ</lastname>  
</employee>
```

```
<xsd:element name="employee">  
  <xsd:complexType>  
    <xsd:sequence>  
      <xsd:element name="firstname" type="xsd:string"/>  
      <xsd:element name="lastname" type="xsd:string"/>  
    </xsd:sequence>  
  </xsd:complexType>  
</xsd:element>
```

Personnaliser un type

```
<employee>  
  <firstname>Moussa</firstname>  
  <lastname>Bâ</lastname>  
</employee>
```

```
<xsd:element name="employee" type="personinfo"/>  
<xsd:complexType name="personinfo">  
  <xsd:sequence>  
    <xsd:element name="firstname" type="xsd:string"/>  
    <xsd:element name="lastname" type="xsd:string"/>  
  </xsd:sequence>  
</xsd:complexType>
```

Personnaliser un type

```
<xsd:element name="employee" type="personinfo"/>
<xsd:element name="student" type="personinfo"/>
<xsd:element name="member" type="personinfo"/>

<xsd:complexType name="personinfo">
  <xsd:sequence>
    <xsd:element name="firstname" type="xsd:string"/>
    <xsd:element name="lastname" type="xsd:string"/>
  </xsd:sequence>
</xsd:complexType>
```



Indicateur de types complexes

- On a 7 types d'indicateur
- Indicateurs de l'ordre
 - Tous (All)
 - Choix (Choice)
 - Séquence (Sequence)
- Indicateurs d'occurrence
 - maxOccurs
 - minOccurs

Indicateur All

- Spécifie que des éléments fils peuvent apparaître en n'importe quel ordre et chaque élément fils doit se produire seulement une fois

```
<xsd:element name="person">
  <xsd:complexType>
    <xsd:all>
      <xsd:element name="firstname" type="xsd:string"/>
      <xsd:element name="lastname" type="xsd:string"/>
    </xsd:all>
  </xsd:complexType>
</xsd:element>
```


Indicateur Choice

- Spécifie qu'on ne peut avoir qu'un élément fils parmi les éléments fils qui peuvent être produits

```
<xsd:element name="person">
  <xsd:complexType>
    <xsd:choice>
      <xsd:element name="employee" type="employee"/>
      <xsd:element name="member" type="member"/>
    </xsd:choice>
  </xsd:complexType>
</xsd:element>
```

Indicateur Sequence

- Spécifie que les éléments fils doivent être produits en ordre

```
<xsd:element name="person">  
  <xsd:complexType>  
    <xsd:sequence>  
      <xsd:element name="firstname" type="xsd:string"/>  
      <xsd:element name="lastname" type="xsd:string"/>  
    </xsd:sequence>  
  </xsd:complexType>  
</xsd:element>
```

Indicateur maxOccurs

- Spécifie le nombre maximum de fois qu'un élément peut être produit

```
<xsd:element name="person">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="full_name" type="xsd:string"/>
      <xsd:element name="child_name" type="xsd:string" maxOccurs="10"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```

Indicateur minOccurs

- Spécifie le nombre minimum de fois qu'un élément peut être produit

```
<xsd:element name="person">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="full_name" type="xsd:string"/>
      <xsd:element name="child_name" type="xsd:string"
maxOccurs="10" minOccurs="1"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```

Autre moyen pour déclarer

```
<xsd:element name="a"> <xsd:complexType>  
  <xsd:sequence maxOccurs="unbounded">  
    <xsd:element ref="b"/>  
    <xsd:element ref="c"/>  
  </xsd:sequence> </xsd:complexType>  
</xsd:element>  
<xsd:element name="b">  
  <xsd:complexType mixed="true">  
    <xsd:attribute name="e" use="required"/>  
  </xsd:complexType>  
</xsd:element>  
<xsd:element name="c" type="xsd:string"/>
```

Définition et utilisation d'un groupe

- Les groupes permettent de déclarer des blocs de sous-éléments.
- **<xsd:group name="PersonneGroup">**
 - **<xsd:sequence>**
 - **<xsd:element name="nom" type="xsd:string"/>**
 - **<xsd:element name="prenom" type="xsd:string"/>**
 - **</xsd:sequence>**
- **</xsd:group>**
- **<xsd:complexType name="MarinType">**
 - **<xsd:sequence>**
 - **<xsd:group ref="PersonneGroup"/>**
 - **<xsd:element name="naissance" type="xsd:date"/>**
 - **</xsd:sequence>**
- **</xsd:complexType>**

Lier le Schéma avec le fichier

- on fait référence au schéma dans le document XML en utilisant l'attribut **xmlns:xsi:noNamespaceSchemaLocation**.
- Par exemple:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<persons xmlns:xsi=http://www.w3.org/2001/XMLSchema-
instance  xsi:noNamespaceSchemaLocation="non_fic.xsd">
  <person>
    .....
  </person>
```

Exemple

- Traduire le plus fidèlement possible, ce schéma de base de données en un document XMLSchéma
 - Client (NumCl, Nom, Prénom, Age, Adresse)
 - Commande (NumCo, DateCo, #NumCl)
 - Produit (IdProd, Libellé, Prix, Provenance)
 - LigneCommande (#NumCo, #IdProd, Quantité)



Chapitre 5

L'extraction de fragments : XPath, XQuery



Introduction

- Comme son nom l'indique, XPath est une spécification fondée sur l'utilisation de chemin d'accès permettant de se déplacer au sein du document XML.
- Dans ce but, un certain nombre de fonctions ont été définies. Elles permettent de traiter les chaînes de caractères, les booléens et les nombres.



Introduction

- XPath est un langage avec une syntaxe non XML, permettant d'adresser les différents noeuds ou groupes de noeuds particuliers d'un document XML.
- XPath voit le document XML comme un arbre de noeuds, qu'il permet de parcourir selon des axes (fils, parent, ancêtre, descendant, ...) et en sélectionnant les noeuds par leur nom.
- Les expressions XPath permettent également de représenter des chaînes de caractères, des nombres et des booléens.

Introduction

- XPath permet d'**effectuer des requêtes** dans un document XML.
- XPath est aux documents XML, ce qu'est SQL aux bases de données.
- XPath est **utilisé par XSLT** pour faire de la transformation de documents XML.

Chemin de localisation

- Un chemin de localisation peut être de type absolu ou relatif.
 - Dans le cas où il est de type **absolu**, il commence toujours par le signe / indiquant la racine du document XML ;
 - Dans le cas où il est de type **relatif**, le noeud de départ est le noeud contextuel courant.

Chemin de localisation

- Un chemin de localisation est composé de **trois parties** :
 - un **axe**, définissant le sens de la relation entre le noeud courant et le jeu de noeuds à localiser;
 - un **noeud** spécifiant le type de noeud à localiser;
 - 0 à n **prédicats** permettant d'affiner la recherche sur le jeu de noeuds à récupérer.

Chemin de localisation

- Par exemple, dans le chemin `child::section[position()=1]`,
 - `child` est le nom de l'axe,
 - `section` le type de noeud à localiser (élément ou attribut)
 - `[position()=1]` est un prédicat.
- Les doubles `::` sont obligatoires.
- La syntaxe d'une localisation s'analyse de gauche à droite.
- Dans notre cas, on cherche dans le noeud courant, un noeud section qui est le premier noeud de son type.

Axes

- **child** : contient les enfants directs du noeud contextuel.
- **descendant** : contient les descendants du nœud contextuel. Un descendant peut être un enfant, un petitenfant...

Axes

- **parent** : contient le parent du noeud contextuel, s'il y en a un.
- **ancestor** : contient les ancêtres du noeud contextuel. Cela comprend son père, le père de son père... Cet axe contient toujours le noeud racine, excepté dans le cas où le noeud contextuel serait lui-même le noeud racine.

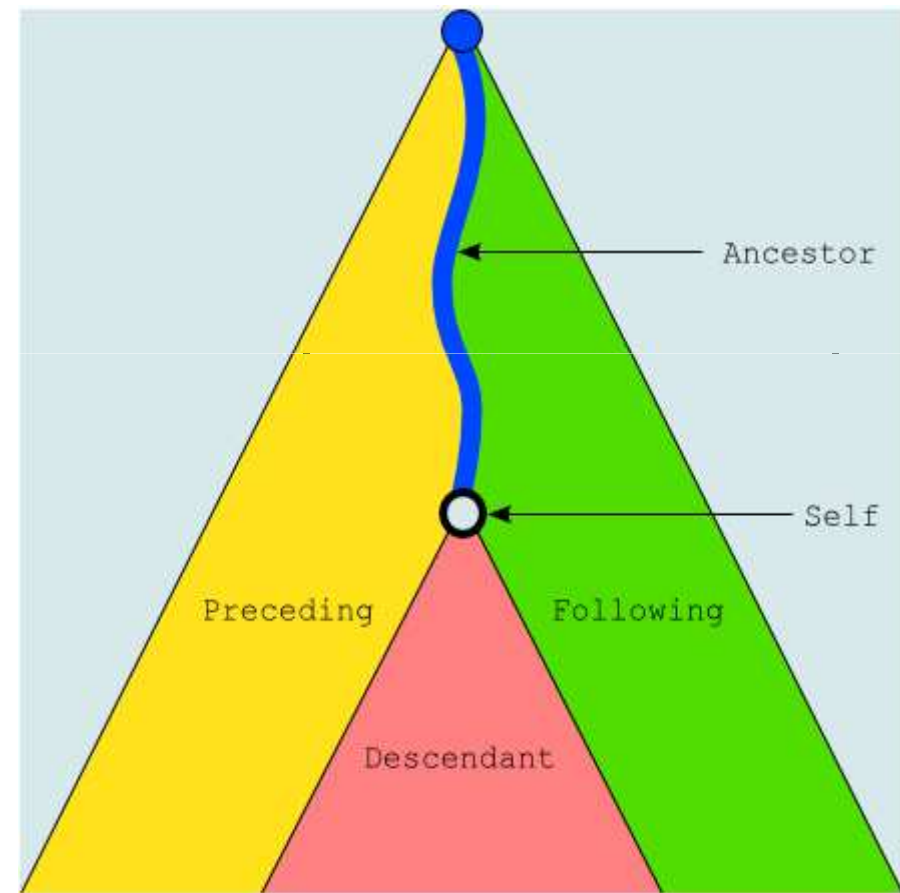
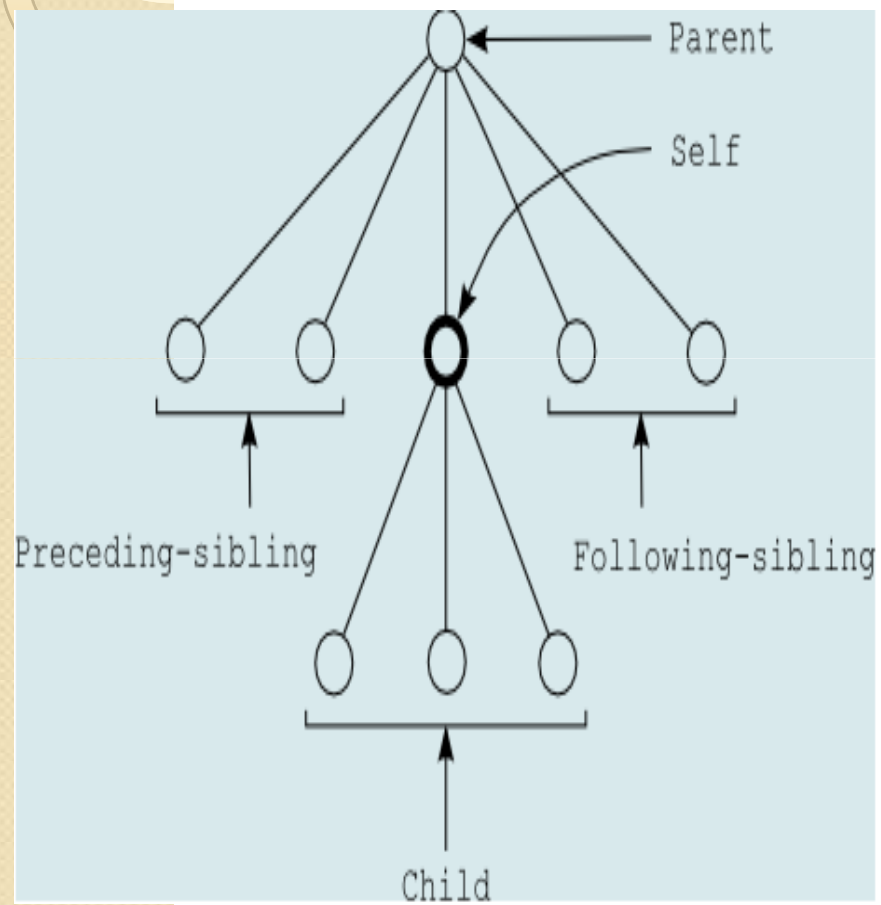
Axes

- **following-sibling** : contient tous les noeuds frères qui suivent le noeud contextuel. Dans le cas où ce noeud est un attribut ou un espace de noms, la cible suivante est vide.
- **preceding-sibling** : contient tous les frères prédécesseurs du noeud contextuel ; si le noeud contextuel est un attribut ou un espace de noms, la cible précédente est vide.

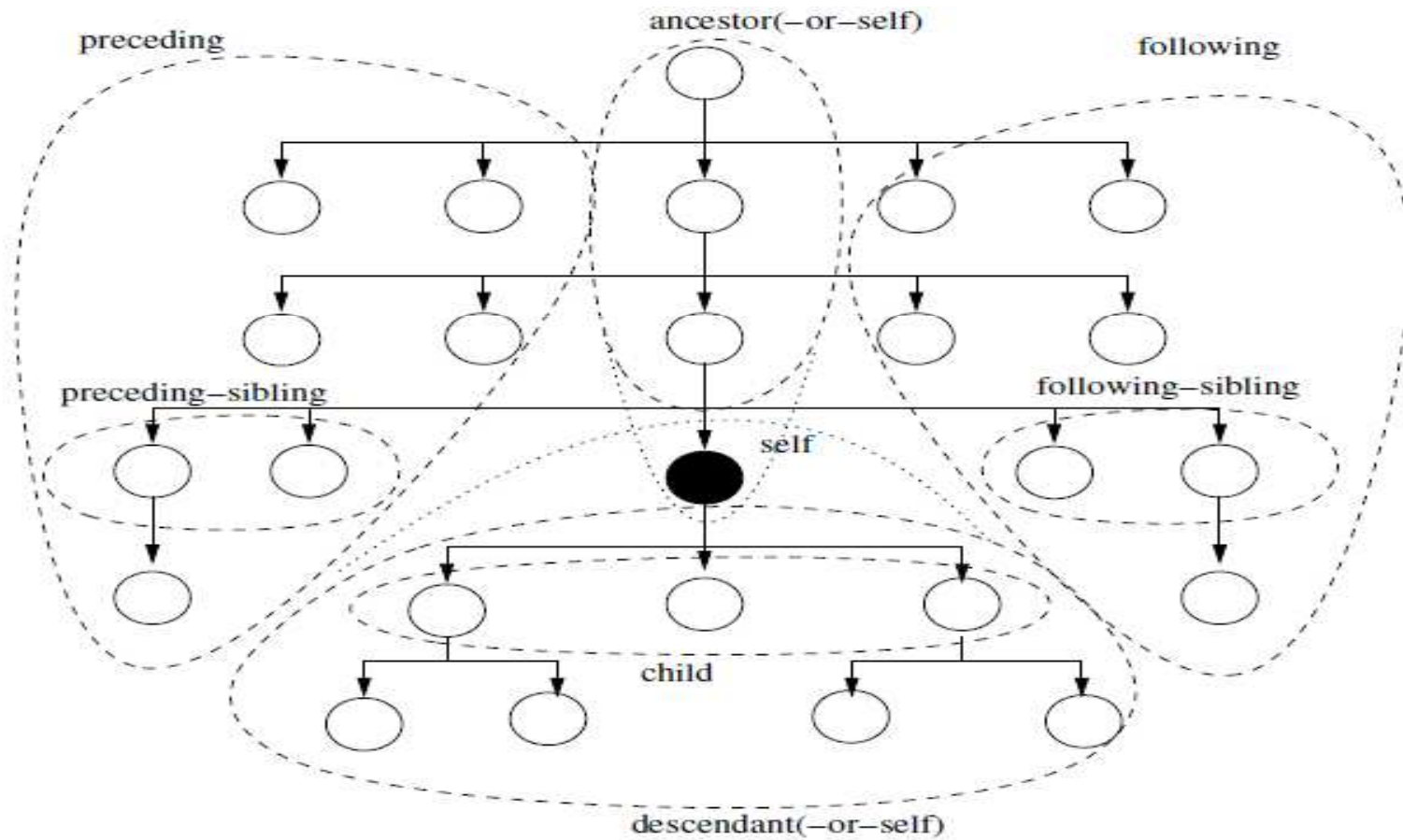
Axes

- **following** : contient tous les noeuds du même document que le noeud contextuel qui sont après le noeud contextuel dans l'ordre du document, à l'exclusion de tout descendant, des attributs et des espaces de noms.
- **preceding** : contient tous les prédécesseurs du noeud contextuel ; si le noeud contextuel est un attribut ou un espace de noms, la cible précédente est vide.

Axes Axes



Axes



Axes

- **attribute** : contient les attributs du noeud contextuel ; l'axe est vide quand le noeud n'est pas un élément.
- **namespace** : contient tous les noeuds des espaces de noms du noeud contextuel ; l'axe est vide quand le noeud contextuel n'est pas un élément.
- **self** : contient seulement le noeud contextuel.
- **descendant-or-self** : contient le noeud contextuel et ses descendants.
- **ancestor-or-self** : contient le noeud contextuel et ses ancêtres. Cet axe contiendra toujours le noeud racine

Prédicats

- Un prédicat **mis entre crochets** permet encore de restreindre l'ensemble des nœuds sélectionnés.
- Les prédicats peuvent combiner avec les opérateurs **or** et **and** plusieurs conditions qui peuvent faire intervenir des fonctions prédéfinies.

Exemples

- **child::text ou text**
 - élément text fils d'un nœud du contexte.
- **/child::text ou /text**
 - élément text sous la racine c'est-à-dire contenant tout le document
- **attribute::text ou @text**
 - attribut text d'un nœud du contexte

Exemples

- **child::chapter/child::section** ou **chapter/section**
 - élément section fils d'un nœud chapter lui-même fils d'un nœud du contexte.
- **parent::node()/child::chapter** ou **../chapter**
 - élément chapter frère d'un nœud du contexte.
 -
- **self::node()/descendant-or-self::section** ou **../section**
 - élément section descendant d'un nœud du contexte.

Exemples

- **child::section[position()=2]** ou **section[2]**
 - deuxième élément section fils du nœud courant.
- **descendant::p/following-sibling::em[position()=1]**
ou **//p/following-sibling::em[1]**
 - premier frère em d'un élément p descendant du nœud courant.
- **child::section[child::title]** ou **section[title]**
 - élément section fils du nœud courant et ayant un fils title.

Exemples

- **child::section[attribute::title]** ou **section[@title]**
 - élément section fils du nœud courant et ayant un attribut title.
- **child::section[attribute::title='Un titre']** ou **section[@title='Un titre']**
 - élément section fils du nœud courant et ayant un attribut title égal à la chaîne Un titre.

Exemples

- **chapter[count(child::section) > 1]**
 - élément chapter ayant plus d'un élément section comme fils.
- **@*[name() != 'id']**
 - attributs autres que l'attribut id.
- **section[2][@type='dual']**
 - deuxième élément section du nœud courant si son attribut type vaut dual.

XQuery

- XQuery est un langage de puissant pour extraire des données XML
- Type de données: un seul "document" ou encore des collections sous forme de:
 - fichiers
 - bases de données XML
 - XML "en mémoire" (abres DOM)

XQuery

- Ex: Une requête qui permet très simplement de renvoyer tous les noeuds **auteur** du document biblio.xml.
 - Aucune mise en forme n'étant pratiquée, les nœuds renvoyés le sont avec leurs balises et leur contenu telqu'il s'exprime dans le document.
 - **document("biblio.xml")//auteur**

Les expressions FLWOR

- FLOWR = "For-Let-Where-Order-Return"
-
- rappelle l'idée du select-from-where-..-order-by de SQL

Document Exemple

• <bibliothèque>

- <livre année="2002">
 - <titre> Prélude à fondation </titre>
 - <auteur><nom> Asimov </nom><prénom> Isaac </prénom></auteur>
 - <éditeur> Pocket </éditeur>
 - <prix>6</prix>
- </livre>
- <livre année="1995">
 - <titre> La marche des millénaires </titre>
 - <auteur><nom> Asimov </nom><prénom> Isaac </prénom></auteur>
 - <auteur><nom> White </nom><prénom> Frank </prénom></auteur>
 - <éditeur> Flammarion </éditeur>
 - <prix>8.45</prix>
- </livre>
- <livre année="2005">
 - <titre> Avant Dune </titre>
 - <auteur><nom> Anderson </nom>
 - <prénom> Kevin J. </prénom></auteur>
 - <auteur><nom> Herbert </nom><prénom> Brian </prénom></auteur>
 - <éditeur> Pocket </éditeur>
 - <prix>12.3</prix>
- </livre>

• </bibliothèque>

Les expressions FLWOR

- **for** = itération sur une liste de fragments xml
- **let** = association du résultat d'une expression à une variable
- **where** = condition de sélection
- **order** = tri des résultats
- **return** = expression à retourner

"for"

- **for \$variable in expression_recherche
RETURN**
- "for" associe à chaque \$variable une valeur (fragment XML) trouvé pour l'expression XQuery
- Exemple:
- **for \$t in
document("biblio.xml")//livre**

let

- **"let"** permet d'assigner une valeur à une variable
- requête qui permet d'affecter la variable **\$b** successivement avec le contenu des noeuds **livre** du document biblio.xml (clause **for**) et d'affecter à **\$a** l'ensemble des nœuds **auteur** contenus dans chaque itération de **\$b** (clause **let**).
- **for \$b in document("biblio.xml")//livre**
- **let \$a := \$b/auteur**
- **return <livre nb_auteurs="{count(\$a)}"> { \$a } </livre>**

where

- permet de définir une condition de sélection
- requête pour extraire le **titre** des livres dont le premier noeud fils **auteur** contient comme valeur de **nom** Mansour.
- **for \$a in document("biblio.xml")//livre**
- **where \$a/auteur[1]/nom eq "Mansour"**
- **return \$a/titre**

order

- permet de trier les résultats
- requête pour extraire le **titre** des livres dont le premier noeud fils **auteur** contient comme valeur de **nom** Mansour.
- **for \$a in document("biblio.xml")//livre**
- **where \$a/auteur[1]/nom eq "Mansour"**
Order by \$a
- **return \$a/titre**

Exemple avec jointure (avec EditiX)

- **Draft de DTD**
- `<!ELEMENT base (etudiant | class)*>`
- `<!ELEMENT etudiant (nom, prenom, adresse)>`
- `<!ELEMENT class (nom)>`
- `<!ATTLIST etudiant identifiant ID classe IDREF>`
- `<!ATTLIST class ident ID>`
- **Nom et prénom des étudiant qui sont en WEB2 (nom classe)**

```
<?xml version="1.0" encoding="UTF-8"?>
<liste>
  <etudiant>
    <nom>KARE</nom>
    <prenom>Massamba</prenom>
  </etudiant>
  <etudiant>
    <nom>DIALLO</nom>
    <prenom>Ndiogou</prenom>
  </etudiant>
</liste>
```

`<liste>`

- for \$a in /base/etudiant
- for \$b in /base/class
- where (\$a/@classe=\$b/@ident and \$b/nom='WEB2')
- return
- `<etudiant>{$a/nom, $a/prenom}`
- `</etudiant>`
- }

`</liste>`



FIN